PaxosKV

A complete LeanFM worked example

Project team

October 28, 2026

Contents

1 What this project is **1**
1.1 Reproducing everything 1

2 Requirements **2**
2.1 Boundary and assumptions 2
2.2 Related grammar set 2
2.3 Two-session interaction projection 3
2.4 Observable safety and progress obligations 4
2.5 Prompt-derived outputs 4
2.6 Web behavior 5

3 Derived views **6**

4 Implementation **10**
4.1 Target and processes 10
4.2 Durable state 10
4.3 Consensus and reads 10
4.4 Web application 11
4.5 Resource choices 11
4.6 Observability 11
4.7 Generated source layout 11
4.8 Conformance tests 12
4.9 Runtime 12

5 Running it **13**

6 Known gaps **14**

Chapter 1

What this project is

PaxosKV is a key/value database replicated across three Paxos replicas, with a one-page web front end. Two browser sessions can write at the same time; each write is acknowledged only after two replicas have durably stored it, and every listing shows one committed prefix of the log.

It is the smallest complete example of the LeanFM method: one book (this one), typed requirements in `spec/Requirements.lean` and `spec/Requirements.proto`, an implementation mapping in `spec/Implementation.lean`, Go source in `source/`, views derived from the requirements in `renders/`, and evidence from real runs in `reports/`. Everything the project owns lives under this one directory.

1.1 Reproducing everything

From the project root, with the LeanFM installation built (`make build` two directories up):

```
make spec-check     # compile and validate the Lean specification
make views          # regenerate renders/ from the requirements
make app-check      # go vet, go test -race
make run            # three replicas + web app on 127.0.0.1:8082
make browser-check  # (second terminal) two concurrent browser sessions
make book           # this book, light and dark
```

Replica data lives in `.cache/data/` and survives restarts; `make -C source clean-data` deletes it.

Chapter 2

Requirements

2.1 Boundary and assumptions

The Paxos participant is `KVReplica`, instantiated as one homogeneous cluster of exactly three members: `kv-0`, `kv-1`, and `kv-2`. `Browser` and `WebApp` are actors but are not Paxos participants. Two independent browser sessions may have tasks in flight concurrently.

The confirmed availability premise means that at every modeled instant at least two replicas are running, all replicas can reach one another while running, and each can reach its durable storage. This gives a live majority. Persistence additionally requires an acceptor to fsync its accepted ballot, slot, key, and value before sending `Accepted`; the web response is not successful until a quorum of two durable acceptances exists. A restarted replica recovers its accepted log before participating. If all three replicas stop, the service is unavailable rather than forgetful: after restart, quorum recovery reconstructs the committed log from durable acceptor state before reads or writes resume.

The confirmed conflict rule is last-committed-slot-wins: if the two browser sessions write the same key concurrently, the leader assigns distinct Paxos log slots and the value in the later committed slot is displayed.

2.2 Related grammar set

```
project paxos_kv ::=
  repeat(quorum_write || linearizable_list || recover_replica)

quorum_write ::=
  Browser.Put(key, value, session, request_id) ;
  WebApp.Propose(key, value, session, request_id) ;
  Leader.Assign(ballot, slot) ;
  parallel(
    Replica.Accept(ballot, slot, key, value) ;
    Replica.Accept(ballot, slot, key, value) ;
    Replica.Accept(ballot, slot, key, value)
  ) ;
  mOfN(2, Replica.DurableAccepted(ballot, slot)) ;
  Leader.Commit(ballot, slot, key, value) ;
  WebApp.PutSucceeded(key, value, slot) ;
  ref(linearizable_list)
```

```
linearizable_list ::=
  Browser.List(session, request_id) ;
  WebApp.ReadBarrier(ballot) ;
  mOfN(2, Replica.ReadBarrierAck(ballot, committed_slot)) ;
  Leader.Snapshot(committed_slot, entries) ;
  WebApp.ListSucceeded(committed_slot, entries)

recover_replica ::=
  Replica.Unavailable(instance) ;
  Replica.Restarted(instance) ;
  Replica.RecoverAcceptedLog(instance) ;
  Replica.CatchUp(committed_slot) ;
  Replica.Available(instance)
```

The notation above is intentionally behavioral. An implementation may use a stable Multi-Paxos leader, but leader election and ballot changes must remain observable whenever leadership changes.

2.3 Two-session interaction projection

This is the interaction structure to be generated from the grammar. The two browser branches are incomparable until the leader assigns slots; either may arrive first.



The durable event representation must keep separate `(session, scenario)` keys and use `priorIds` to record immediate predecessor order. `priorIds = []` means that an event is a minimal (first) event in the scenario. If several events have no predecessor, they are mutually unordered and may proceed in parallel. A singleton list is the ordinary prior-event link; a list with several IDs is a join. These links are ordering backpointers, not claims of causation. Event file line order must not serialize otherwise unordered browser requests.

2.4 Observable safety and progress obligations

- A successful PUT necessarily has two prior `DurableAccepted` observations for the same ballot, slot, key, and value.
- No two different values are committed for the same log slot.
- Every committed slot remains present after any one replica is unavailable.
- A list result reflects one committed prefix selected after a quorum read barrier; it does not combine entries from different prefixes.
- With two mutually reachable durable replicas and a leader eventually chosen, every accepted browser request eventually succeeds or returns an explicit error.
- Replica unavailability, leadership loss, retry, rejection, and recovery are visible outcomes rather than hidden timeouts.
- At most three replica instances exist, all implementing the same `KVReplica` actor specification.

2.5 Prompt-derived outputs

`paxos.write.interaction` (interaction diagram) How does a browser write become persistent? *Reducer:* Partition write events by `(session,request_id)` and render quorum branches in `priorIds` order, with typed message fields in each node

`paxos.write.state_machine` (state machine) Which observable write states and failure exits exist? *Reducer:* Render residuals of `quorum_write`

`paxos.list.interaction` (interaction diagram) How is the complete key/value listing made linearizable? *Reducer:* Render list and read-barrier events by `(session,request_id)`

`paxos.list.state_machine` (state machine) Which observable list states and failure exits exist? *Reducer:* Render residuals of `linearizable_list`

`paxos.concurrent.order` (prior-pointer graph) Can two browser sessions access the app concurrently? *Reducer:* Render incomparable request branches, slot assignment, quorum joins, and responses

`paxos.commit_latency` (2D function) How does write latency change with concurrent browser sessions? *Reducer:* Pair PUT start to success/error by prior ID; x=max concurrent sessions, y=latency milliseconds

`paxos.quorum_size` (scalar and histogram) Did every acknowledged write have a durable majority? *Reducer:* Count distinct accepting replica instances per committed slot

`paxos.replica_availability` (line) Were at least two replicas available throughout the observation window? *Reducer:* Sweep `Unavailable`/`Available` intervals and plot available replica count over monotonic time

`paxos.recovery_lag` (2D function) How far behind is a restarting replica? *Reducer:* x=time since restart; y=leader committed slot minus replica applied slot

`paxos.outcomes` (pie/histogram) What happened to browser PUT and list requests? *Reducer:* Count success, explicit rejection, timeout/censored, and leadership-loss outcomes

Numeric reductions use these stable output IDs at `/metrics`. Every diagram, state machine, and chart is derived from the accepted grammar and events rather than maintained as a second specification.

2.6 Web behavior

The page contains a string `key` input, a string `value` input, and a submit action. Below the form it renders the linearizable listing of all committed key/value pairs. After a successful write the initiating session receives a page containing a listing at least as new as its committed write. The other browser session observes that write on its next list operation; server push is not currently required.

There is no authentication and no delete operation. Keys are nonempty UTF-8 strings of at most 128 encoded bytes. Values are UTF-8 strings of at most 4096 encoded bytes. The database holds at most 10,000 distinct keys and supports at least two browser sessions with requests in flight concurrently. Invalid or capacity-exceeding writes return explicit errors without changing the committed log. Each successful submission returns a listing containing at least that committed write.

Chapter 3

Derived views

Every figure in this chapter is generated by `spec/Views.lean` from `Project.Requirements.requirements`; none is drawn by hand.

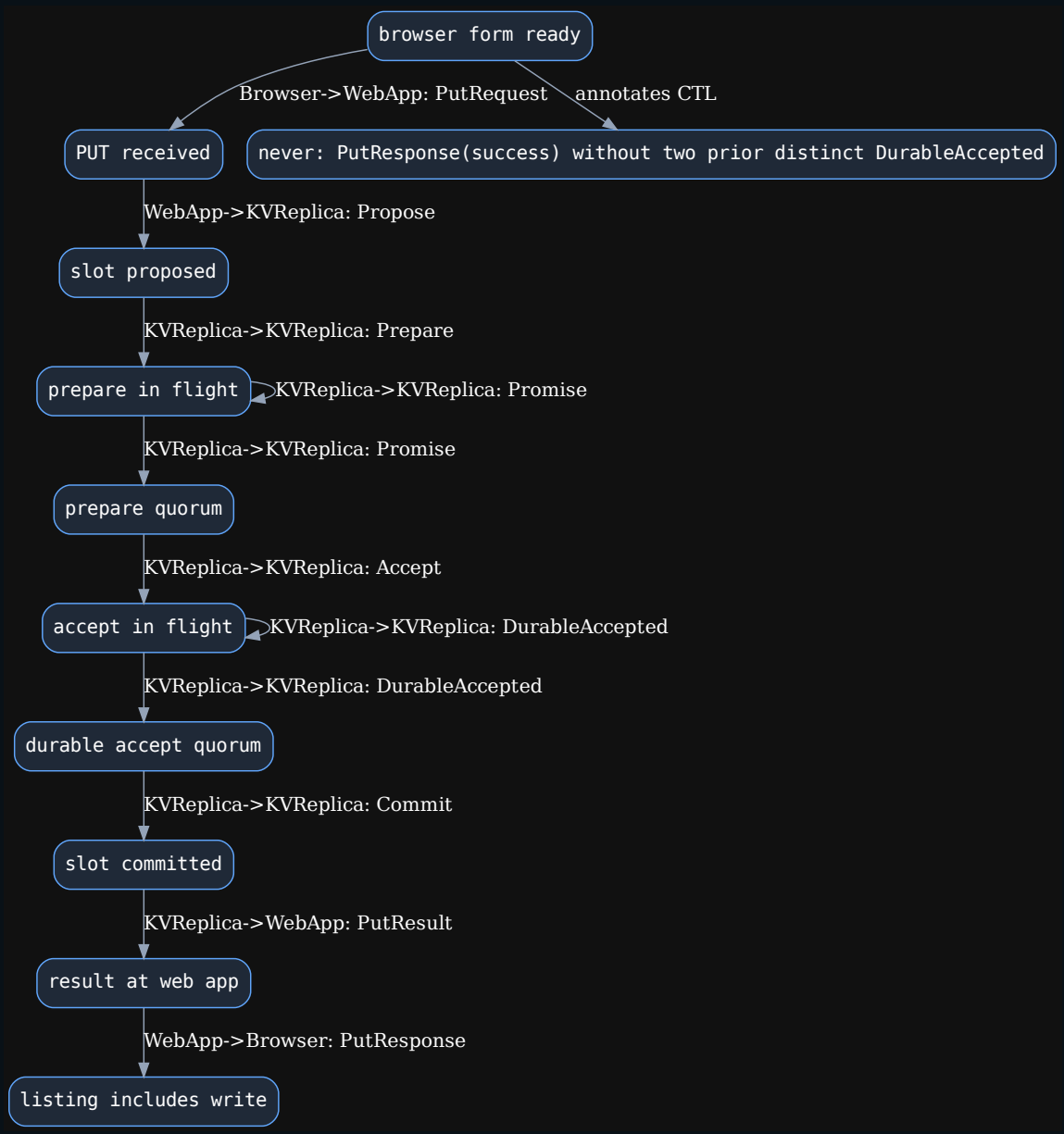


Figure 3.1: Task `quorum_write`: states and transitions.

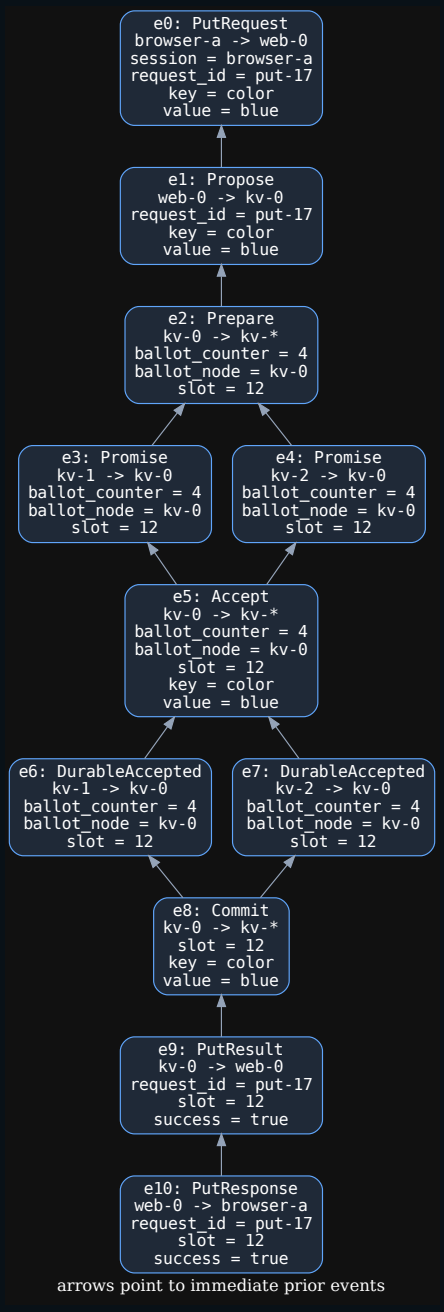


Figure 3.2: One concrete quorum write as a prior-pointer graph. The two `Promise` events fork from `Prepare`; `Accept` and `Commit` are two-of-three joins.

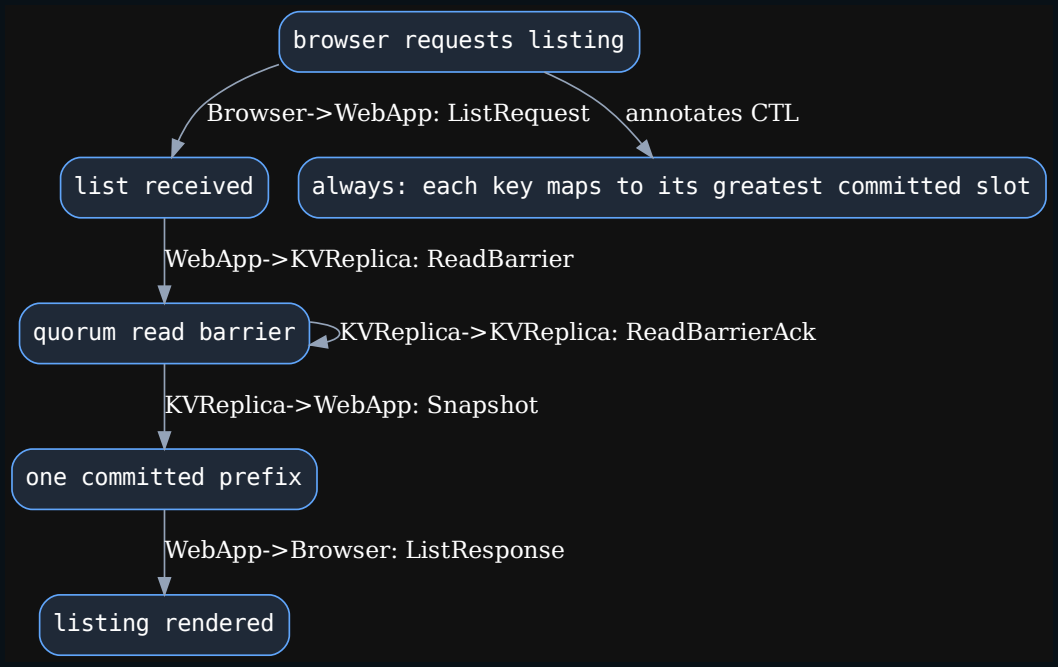


Figure 3.3: Task `linearizable_list`.

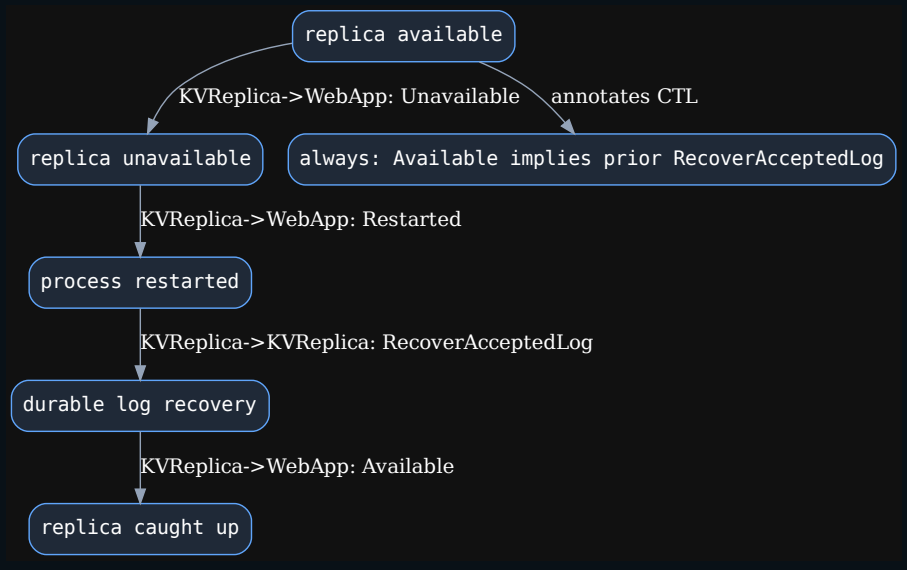


Figure 3.4: Task `recover_replica`.

Chapter 4

Implementation

This chapter maps the requirements to an implementation. It does not change their observable meaning.

4.1 Target and processes

- Target language: Go.
- One `paxos-kv` executable with subcommands `web` and `replica`.
- One WebApp process and three independently started replica processes.
- Replica IDs and addresses are supplied explicitly; membership is fixed at `kv-0`, `kv-1`, and `kv-2`.
- Browser traffic uses HTTP. Replica protocol traffic uses length-delimited protobuf messages over TCP connections.

4.2 Durable state

Each replica owns a separate data directory containing an append-only accepted log and a small metadata file for the highest promised ballot. Every update is written to a temporary record, flushed with `fsync`, and atomically made visible before `DurableAccepted` is sent. Recovery scans and validates the log, truncates only an incomplete final record, and reconstructs accepted slots. No process shares a replica data directory.

The leader records commit decisions durably. On startup or leadership change it runs Paxos recovery against a quorum before accepting HTTP work, choosing the accepted value with the highest ballot for each slot and filling any gaps.

4.3 Consensus and reads

- Stable-leader Multi-Paxos assigns monotonically increasing log slots.
- Ballots are ordered pairs `(counter, replica_id)`.
- A candidate must receive promises from two replicas before becoming leader.
- A value is committed after two distinct replicas have durably accepted the identical `(ballot, slot, key, value)` tuple.
- Repeated messages are idempotent by `(ballot, slot)` and browser `(session_id, request_id)`.

- The materialized key/value map applies committed slots in increasing order; this implements last-committed-slot-wins.
- A list request uses a quorum read barrier and returns one applied committed prefix. A successful PUT waits until its slot is applied, then obtains the listing from a prefix at least that new.

4.4 Web application

- `GET /` obtains a linearizable listing and renders the form followed by the sorted key/value table.
- `POST /put` accepts form fields `key` and `value`, validates encoded byte limits, submits the write, and renders the resulting listing.
- A secure random cookie supplies `session_id`; every request gets a unique `request_id`.
- HTML is escaped. Request bodies, response bodies, and internal queues are bounded. Two browser sessions may progress concurrently.
- Failures return an explicit page and appropriate HTTP status; they never display an uncommitted value as successful.

4.5 Resource choices

- Web request body: 8 KiB maximum.
- WebApp: 64 in-flight requests and a bounded 128-item submission queue.
- Each replica: bounded 256-message inbound queue and 64 simultaneous peer operations.
- Database: 10,000 distinct keys, with limits enforced before proposal.
- Timeouts cancel a client attempt but do not erase a value already accepted; retries use the same request ID and discover the committed result.

4.6 Observability

Every protocol transition is meant to emit the LeanFM event envelope with event ID, repeated predecessor IDs, browser session, task/request ID, concrete source and destination replica IDs, monotonic time, and protobuf atom. `/metrics` exports commit latency, quorum size, available replica count, recovery lag, request outcomes, queue depth, and memory headroom under the accepted output IDs. Logs include ballot and slot but never unescaped HTML.

4.7 Generated source layout

```
source/Makefile                      build, run, stop, test
source/runtime.py                    Runtime actor (supervisor)
source/browser_check.py              two-session browser check
source/cmd/paxos-kv/main.go          replica and web subcommands
source/internal/web/app.go           WebApp and Browser bindings
source/internal/paxos/node.go        KVReplica: Multi-Paxos and recovery
source/internal/paxos/storage.go     fsynced accepted log
source/internal/protocol/            wire Envelope (protobuf)
source/internal/metrics/metrics.go   /metrics
```

`source/internal/trace/events.go`	`events/kv-N.jsonl`
`source/web/templates/index.html`	the page
`source/traceability.json`	requirement -> file map

4.8 Conformance tests

- Two browser sessions write different keys concurrently and both listings converge to the same committed map.
- Two sessions write one key concurrently; the later committed slot wins.
- Every HTTP success has two prior durable acceptances from distinct replicas.
- Stop any one replica during writes; the other two continue and the stopped replica catches up after restart.
- Stop all replicas after acknowledged writes; restart two and recover every acknowledged value before serving.
- Corrupt or truncate the final log record and verify safe recovery without inventing a commit.
- Reject oversized, empty-key, and 10,001st-key writes without mutation.
- Reject conflicting values for one slot, stale ballots, malformed protobuf, and illegal trace order.
- Exercise full queues, timeouts, idempotent retries, and HTML escaping.

The observability section above is the target. Today the replicas emit only terminal events; see the Known gaps chapter.

4.9 Runtime

`source/runtime.py` is the `Runtime` actor: it starts `kv-0`, `kv-1`, `kv-2` on ports 9100–9102 and the web app on 8082, records its own process identity, and stops only that process. Each replica appends its observed events to `events/kv-N.jsonl`.

Chapter 5

Running it

`source/browser_check.py` opens two independent browser sessions against a running deployment. Session A writes `color=blue` while session B writes `shape=circle` concurrently; A then overwrites `color=green`. Both sessions then list the database. The screenshots below are produced by that script; `reports/browser/browser-check.json` records the rows each session saw.

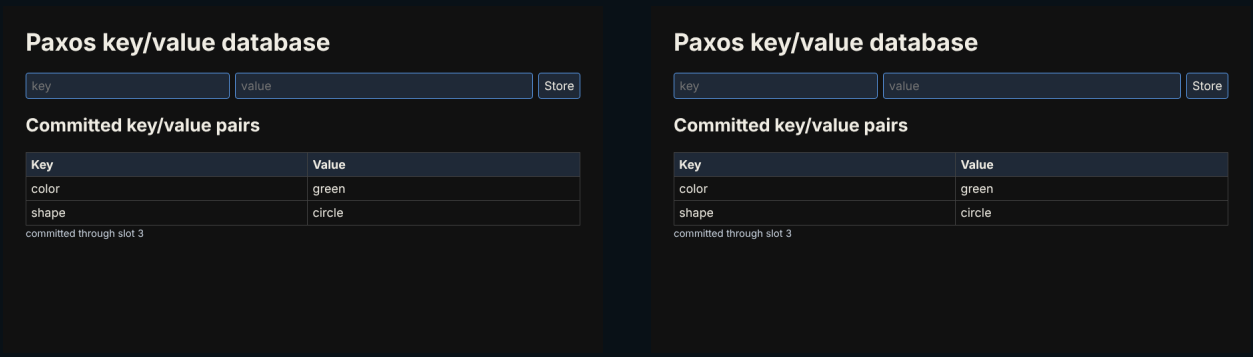


Figure 5.1: Session A (left) and session B (right) after the writes. Both show the same committed map; the later slot for `color` wins.

Chapter 6

Known gaps

This example runs end to end, but it does not yet close the loop from a running system back to the checker. These are the open items, in the order they should be fixed:

1. The protobuf message types in `spec/Requirements.proto` have no fields; the replicas send one generic `Envelope`. The typed messages in the requirements should become the wire format.
2. Replicas log only terminal events (`PutSucceeded`, `ListSucceeded`) and record no `priorIds`. Every send and receive in the grammar should be logged with its predecessors, so a run can be replayed through `check_trace` and the CTL checker.
3. The interaction diagram for a run should then be drawn from `events/` rather than from the hand-written example scenario.
4. The requirements declare three `WebApp` instances; the runtime starts one.
5. Go tests cover envelope round trips and truncated frames, not message loss, duplication, delay or reordering.